

Safety, Speed and Cost

How we stop the AI being wrong, how fast the system is, and what we just fixed

16 September 2026
300 automated tests
5 commits

The short version. We have safety checks that catch the AI making mistakes. We discovered **none of them had ever been tested** — so we had no idea whether they worked. We tested them, and then deliberately broke each one to prove the tests would notice.

We also made the system **about 9 seconds faster** per question, and found that our own quality test was **demanding the wrong answer**.

Part 1 — The five things this document covers

Think of the AI as a **brilliant but occasionally careless assistant** you have hired. Everything below is about managing that person.

TOPIC	WHAT IT MEANS	IN OUR SYSTEM
System prompt	The job description you hand them. "You are a librarian. Only use these books. Always say where you got it."	A 1,169-word instruction sheet, sent with every question
Guardrails	Checking their work <i>before</i> it goes to the customer.	Five automatic checks, plus one that can refuse to publish a report entirely
Rules	The policies they must follow. Some you <i>ask</i> for. Some you <i>enforce</i> .	14 asked for in the job description; the critical ones enforced in code
Latency	How long you wait for an answer.	Under 1 second for three agents. About 28 seconds for the one that uses AI
Efficiency	What each answer costs, in money and memory.	Every AI question is a paid call. The other three agents are free

The single most important idea in this whole document:

Asking the AI to behave gives you "usually". Enforcing it in code gives you "always."

We measured this. Told to copy a formula exactly, the AI got it right **2 times out of 5**. With the same rule enforced by code instead, **5 out of 5**.

Part 2 — What we found, and fixed

FOUND None of the safety checks had ever been tested

The system has five checks that catch the AI getting things wrong, plus a fact-checker that can refuse to publish a report. That is the code standing between the AI and a wrong number reaching an engineer.

Not one of them had a single test. Nobody had ever confirmed any of it worked.

What we did

Wrote **38 new tests** that deliberately try to break them. Each one feeds a check exactly the mistake it exists to catch — using real errors we had actually seen:

- The AI "tidying up" a formula and quietly dropping a divide-by-two — which **doubles every answer calculated from it**
- The AI inventing a standard's part number when the book had only said "the standard"
- The AI stating a vibration limit it was measured getting right **1 time in 5**

Then we broke them on purpose

A test that passes proves nothing on its own. So each check was **switched off, one at a time**, and everything re-run:

Safety check switched off	Did the tests notice?
formula guard	CAUGHT
invented-standard guard	CAUGHT
vibration-limit guard	CAUGHT
referral guard	CAUGHT
formula caveat	CAUGHT

All five. If anyone deletes one tomorrow, a test fails immediately.

This mattered because **this project has already shipped two checks that could never fail.** They passed everything while looking like they were working. We did not want a third.

FOUND Our own quality test demanded the wrong answer

This was the most serious finding, and it had nothing to do with speed.

The test that checks whether the AI invents things had this rule written into it:

*"The answer must say **2.8**. If it says **4.5**, that is the wrong machine group."*

That is backwards. The international standard grades pumps by whether the motor is separate or built-in — *not* by how powerful the pump is. For a 55 kW pump with a separate motor, **4.5 is correct and 2.8 is the wrong group.**

So the test demanded the wrong answer and rejected the right one.

A correct system would have failed it. A broken one would have passed. That is worse than having no test at all — it actively pushes you toward the bug.

Why it was so hard to spot

Two different machine groups **share the same set of numbers**:

```
Group 1 and Group 3 → 2.3 / 4.5 / 7.1  
Group 2 and Group 4 → 1.4 / 2.8 / 4.5
```

Pick the wrong group and you *still get a plausible-looking number*. Nothing appears broken. That is how the mistake survived in three separate places at once.

The proper fix

Correcting the test was not enough — the wrong idea was also written in plain English in two other files, ready for the next person to copy. So:

1. **The actual calculation was always correct** and already had a test. Good news: the working code was never wrong.
2. **Corrected both explanations**, and pointed them at the one function that holds the rule rather than restating it. A rule written in two places is a rule that can disagree with itself.
3. **Tests now read the numbers from the standard's own table** instead of retyping them. This is the part that prevents it happening again: the test got it wrong *because a human typed a number into it*. Now the only place a number is typed by hand is the single file that transcribes the published standard.
4. **Added a test that makes the rule impossible to misread**: a 5 kW, a 55 kW and a 5 MW pump must all give the same group.

Proven, not assumed. We put the bug back in — changed the code to grade pumps by power again, exactly as the old belief described. Both sets of tests caught it. Then we restored the file and confirmed it was byte-for-byte identical.

FOUND The instruction sheet argued with itself

The AI's job description contained two rules that contradicted each other:

```
Rule A: "Never read a vibration limit out of a book."  
Rule B: "Here is how to correctly quote a vibration limit from a table..."
```

Rule B was left over from before limits moved to their own agent. **A rule arguing with the rule above it is worse than a long list** — the AI has to guess which one you meant.

Deleted. The sheet went from 1,359 to 1,169 words, about **250 fewer words on every single AI call**.

Checked afterwards with the full quality test: 21 of 23 clean. The one remaining miss is a citation pointing at a slightly off-topic passage, and it predates this work.

Part 3 — Speed

We measured before optimising, and it was not what we expected

The assumption was "the AI is slow". **It was not**. Asking two questions in the same session showed where the time really went:

WHAT	TIME	IS IT WORK?
Loading the search model	~12 s	No — startup
Loading the ranking model	~7 s	No — startup
Loading all 8 books	0.34 s	Irrelevant
Searching	~6 s	Yes
The AI thinking	~8 s	Yes

Nearly 20 of the 26 startup seconds were two models loading one after the other, before any thinking happened at all. Had we "optimised the AI" as planned, we would have spent days shaving seconds off the wrong thing.

Two fixes

The two models now load side by side. They do not depend on each other, so there was no reason to queue them. They also start during a moment the system was already waiting on the network — free time that was being wasted.

The searches now run side by side too. Four independent searches were happening one after another. Measured on identical inputs: **13.3 seconds became 8.5 — 36% faster.**

QUESTION	BEFORE	AFTER	SAVED
First question of a session	37.6 s	28.5 s	9.1 s
Every question after that	11.8 s	10.4 s	1.4 s

What we deliberately did not do

About 11 seconds remain, and it is one large model loading from disk. **That cannot be tweaked away.** Every time the command runs it is a fresh start.

Fixing it properly means keeping something running in the background holding the model in memory — a real architectural change, worth doing only if people ask questions often. For an occasional terminal tool it is not yet justified. **Flagging it rather than building it.**

Part 4 — Where the four agents stand

AGENT	SPEED	COST	USES AI?
Limits checker	instant	free	No — pattern matching and a lookup
Data fetcher	0.6 s	free	No
Report writer	0.7 s	free	No
Librarian	28 s	paid	Yes

Three of the four use no AI at all, which is why they are instant, free and cannot invent anything. That is not a limitation — it is the design.

The strongest guardrail of all is **not using AI for the part that must be right**.

Three of the four use no AI at all, which is why they are instant, free and cannot invent anything. That is not a limitation — it is the design.

The strongest guardrail of all is **not using AI for the part that must be right**.

These five topics are not five separate problems. They are one set of trade-offs:

```
More safety checks → slower, and costs more
Longer instructions → better behaviour, more cost per question
Removing the AI → instant, free, never wrong
                  ...but understands far less
```

The limits checker sits at one extreme: no AI, so nothing to guard, nothing to pay, never wrong — but it only handles one narrow job. The librarian sits at the other: real understanding of open questions, but slow, costly, and needing five checks watching it.

Neither is better. They are the right answer for different jobs.

ITEM	BEFORE	AFTER
Tests on the safety checks	0	38
Safety checks proven to fail when broken	none	all 5
Quality test correctness	demanded the wrong answer	corrected and pinned
Contradictory instructions	2 rules in conflict	resolved
First question	37.6 s	28.5 s
Later questions	11.8 s	10.4 s
Total automated tests	260	300

The lesson worth carrying forward. Our quality test was wrong because a person typed a number into it. Every number a test asserts is a place a wrong idea can hide.

So: tests should *derive* their facts from one authoritative source, and that source should be checked against the published document exactly once. And no check should be trusted until it has been made to fail on purpose.

The lesson worth carrying forward. Our quality test was wrong because a person typed a number into it. Every number a test asserts is a place a wrong idea can hide.

So: tests should *derive* their facts from one authoritative source, and that source should be checked against the published document exactly once. And no check should be trusted until it has been made to fail on purpose.

Safety, speed and cost · vibration monitoring agents · 16 September 2026.

Every figure here was measured on the running system at the time of writing, not estimated. Five commits on `main` at github.com/Atharvsk001/VibrationMonitoring.